ULB UNIVERSITÉ LIBRE DE BRUXELLES

EPFC

PRW : Projet de Développement Web

02 - Compléments PHP (8.2)

Classes et objets

Accès aux bases de données via la librairie PDO

Validation des données et injection de code

Sessions et redirections

Upload et stockage des mots de passe

EPFC-ULB

Introduction aux classes et aux objets

Classes et objets

Une **classe** peut être vue comme un nouveau type de données (par exemple, un utilisateur, un message sur un forum, …).

Un **objet** est l'instance d'une classe et a une existence précise (l'utilisateur "benoit", le message "x" publié sur le forum "y" à la date "d", …)

Une classe définit la structure et le comportement de chacun de ses objets.

Elle est composée de variables (**attributs** ou propriétés) et de fonctions (**méthodes**).

Dans les slides suivants, nous introduisons quelques concepts nécessaires à leur compréhension. Plus de détails viendront plus tard et au cours de Programmation Orientée Objet.

Classes, objets et attributs

```php
class User
{
    public $name, $password;
}

$pierre = new User();
$pierre->name = "Pierre";
$pierre->password = "1234";

$bruno = new User();
$bruno->name = "Bruno";
$bruno->password = "5678";

echo "Nom : ".$pierre->name."<br>";
```

Définition d'une classe avec deux attributs

Création d'un objet (mot clé new) via un constructeur par défaut : Ses deux attributs ne sont pas encore initialisés.

Assignation de valeurs aux attributs (remarquez l'usage du symbole ->). Un attribut appartient à un objet.

Lecture d'un attribut

Classes, objets et méthodes

```php
class User
{
    public $name, $password;

    function prettyPrint(){
        echo "User $this->name";
    }
}

$pierre = new User();
$pierre->name = "Pierre";
$pierre->password = "1234";

$pierre->prettyPrint();
```

Définition d'une méthode. Une méthode porte sur l'objet courant (`$this`) et est commune à tous les objets d'une classe.

Appel d'une méthode. Une méthode s'appelle sur un objet (sur une instance d'une classe).
On parle d'envoi de message.

Construction d'un objet

```php
class User
{
    public $name, $password;

    function __construct($aName, $aPassword){
        $this->name = $aName;
        $this->password = $aPassword;
    }

    function prettyPrint(){
        echo "User $this->name";
    }
}

$paul = new User("paul","abcd");

$paul->prettyPrint();
```

Définition d'un constructeur. Un constructeur est une méthode spécifique appelle lors de la construction d'un objet. Lorsque l'on définit un constructeur, le constructeur par défaut (sans arguments) n'est plus accessible.

Création d'une instance de User via un constructeur à deux paramètres.

Fonctionnalités modernes de PHP 8.x (1)

```php
class Date {
    public int $day;
    public int $month;
    public int $year;
}

class User {
    public string $name;
    public Date $dateOfBirth;
    public array|bool $friends;

    public function getAge() : int {
        return 42;
    }

    public function setName(string $name) : void {
        $this->name = $name;
    }

    public function getFriends() : array|bool {
        return $this->friends;
    }
}
```

Constatez que

- les attributs peuvent avoir un type,
- les types peuvent être unis (`array|bool`) et que
- le type des paramètres et le type de retour des fonctions peuvent être spécifiés.

Une `TypeError` se produit à l'exécution en cas de non-respect des types.

Fonctionnalités modernes de PHP 8.x (2)

Les deux morceaux de code suivants font la même chose. Constatez que le second est plus court.

```php
class Date {
    public int $day;
    public int $month;
    public int $year;

    public function __construct(int $day, int $month, int $year){
        $this->day = $day;
        $this->month = $month;
        $this->year = $year;
    }
}
```

```php
class Date {
    public function __construct(public int $day,
                                public int $month,
                                public int $year){
    }
}
```

Fonctionnalités modernes de PHP 8.x (3)

- Les derniers paramètres d'une fonction peuvent avoir une valeur par défaut et donc devenir optionnels.
- On peut nommer les arguments lors de l'appel d'une fonction : dans ce cas, l'ordre n'a plus d'importance.
- Il existe une fonction magique `__toString()` que l'on peut redéfinir comme en Java.

```php
class User{
    public string $name;
    public string $password;

    public function __construct(string $name,
                                string $password = "") {
        $this->name = $name;
        $this->password = $password;
    }

    public function __toString(): string {
        return $this->name;
    }
}

$u = new User(password: "1234", name: "Boris");
$u = new User(name: "Boris", password: "1234");
$u = new User("Boris", "1234");
$u = new User("Boris");

echo $u; //produit "Boris"
```

Raccourci d'écriture et expression `match`

Raccourci d'affichage (*Short echo tag*)

La balise `<?= ?>` est un raccourci syntaxique pour `<?php echo ... ; ?>`.

Elle est particulièrement utile dans les pages où l'on mélange de l'HTML et du PHP, pour insérer des variables de manière plus lisible et concise.

Il n'est pas nécessaire de mettre un point-virgule à la fin de l'instruction avant la balise fermante.

```php
<?php $prenom = "Alice"; ?>

<!-- Avant (Syntaxe classique) -->
<h1>Bonjour <?php echo $prenom; ?> !</h1>

<!-- Maintenant (Syntaxe raccourcie recommandée) -->
<h1>Bonjour <?= $prenom ?> !</h1>
```

L'expression `match` (Alternative à `switch`)

Introduite avec PHP 8, `match` est une évolution plus stricte et plus courte du `switch`.

C'est l'équivalent de l'expression `switch` introduite dans Java 14.

Avantages :

- Renvoie directement une valeur (c'est une expression).
- Utilise une comparaison stricte (===).
- Pas besoin du mot-clé `break`.

```php
$role = 'admin';

// Le résultat de match() est directement stocké dans la variable
$message = match ($role) {
    'admin'  => 'Accès total',
    'editor' => 'Accès en écriture',
    'user'   => 'Accès en lecture',
    default  => 'Accès refusé',
};

echo $message; // Affiche : Accès total
```

Accès aux bases de données en PHP via PDO

PDO : PHP Data Objects

PDO est une extension PHP qui permet d'utiliser une base de données en s'affranchissant du type de SGBD (Oracle, MySQL, …).

Il existe d'autres librairies comme mysqli qui ne permettent que d'utiliser un SGBD spécifique.

Pour se connecter à une base de données, il faut créer un objet PDO en passant à son constructeur une chaîne de connexion (*type, hôte et nom de base de données*), un nom d'utilisateur et un mot de passe :

```php
$pdo = new PDO("mysql:host=localhost;dbname=social;charset=utf8mb4", "root", "root");
```

On peut ensuite éventuellement configurer cette connexion pour, par exemple, déclencher des exceptions si une erreur se produit plus tard (une erreur à l'exécution d'une requête, …). Nous le ferons systématiquement.

```php
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

Qu'est-ce qu'une exception ?

PDO peut déclencher des exceptions quand une erreur se produit (impossibilité de se connecter à une base de données, erreur de syntaxe dans une requête SQL, erreur à l'exécution d'une requête, …

Une exception est une erreur qu'il est possible de "rattraper".

```php
try {
    $pdo = new PDO("mysql:host=localhost;dbname=social;charset=utf8mb4", "root", "root");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (Exception $e) {
    echo $e->getMessage();
}
```

Dans cet exemple, si le constructeur de PDO déclenche une exception, le flux "saute" dans le bloc catch. Sinon, le flux continue.

Le code pouvant déclencher une exception peut se trouver dans un bloc `try`. Si une exception est déclenchée dans ce bloc, l'exécution du bloc est interrompue et saute dans le bloc `catch` associé. Un objet de type `Exception` est reçu par ce bloc et contient de l'information à propos de l'erreur (message, fichier, ligne…). *Nous en reparlerons dans le cours de Programmation Orientée Objet.*

Exemple complet d'interrogation


```php
try {
    $pdo = new PDO("mysql:host=localhost;dbname=social;charset=utf8mb4", "root", "root");

    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $statement = $pdo->query("SELECT pseudo, password FROM Members");

    $array = $statement->fetchAll();

    foreach($array as $row) {
        echo $row["pseudo"] . "<br>";
    }
} catch (Exception $e) {
    die("Une erreur est survenue : " . $e->getMessage());
}
```

`$pdo->query("...")` exécute une requête et renvoie le résultat (un *PDOStatement*)

`$statement->fetchAll()` convertit le résultat en un tableau à deux dimensions : un tableau par ligne.

Il existe aussi `$statement->fetch()` qui renvoie un tableau (une ligne) à chaque appel et renvoie FALSE à la fin (ou si la requête ne donne aucune ligne)

Requêtes préparées

Une requête préparée a l'avantage d'être plus sûre (protection contre les injections de code) et est aussi plus rapide si la requête est exécutée plusieurs fois. Nous vous conseillons donc de les utiliser systématiquement.

Une première façon de préparer et d'exécuter une telle requête est d'utiliser des marqueurs "?" et un tableau passé à la méthode `execute()` :

```php
$statement = $pdo->prepare("SELECT password FROM Members WHERE pseudo = ?");

$statement->execute(["boris"]);

$array = $statement->fetchAll();
```

Si plusieurs marqueurs positionnels "?" sont utilisés, le tableau contiendra plusieurs éléments, dans le même ordre que les marqueurs.

Requêtes préparées avec marqueurs nommés.

Une façon **plus élégante et recommandée** de procéder est d'utiliser des marqueurs nommés composés du symbole ":" et d'un nom.

```php
$statement = $pdo->prepare("SELECT produit
                            FROM produits
                            WHERE categorie = :cat
                              AND prix <= :max");

$statement->execute(["cat" => "fruits", "max" => 2.5]);

$array = $statement->fetchAll();
```

Dans ce cas, la méthode `execute()` reçoit un tableau associatif qui contient les paires `marqueur => valeur`.

Remarques

- Les modifications (`UPDATE`), insertions (`INSERT`) ou suppressions (`DELETE`) fonctionnent exactement de la même façon :

```php
$statement = $pdo->prepare("INSERT INTO Members (pseudo, password)
                            VALUES (:pseudo, :password)");

$statement->execute(["pseudo" => "Paul", "password" => "1234"]);
```

- N'oubliez pas d'exécuter toutes ces instructions dans un bloc `try` : en cas d'erreur, une exception est déclenchée et vous pourrez alors choisir un comportement dans le bloc `catch` au lieu qu'un message d'erreur ne soit produit.

- Une fois une requête exécutée correctement, un premier appel à `fetch()` renverra `FALSE` si aucun résultat n'est retourné et un appel à `fetchAll()` renverra un tableau vide (`count($array)===0`).

Faire la première partie du tutoriel (3a).

Opérateurs liés aux valeurs nulles

L'opérateur "*Null Coalescing*" (??)

Permet de vérifier si une variable existe et n'est pas `null`.

S'il existe, il renvoie sa valeur. Sinon, il renvoie une valeur par défaut.

Très pratique pour traiter les formulaires ($_GET, $_POST) .

```php
// Ancienne méthode (avec isset et opérateur ternaire)
$pseudo = isset($_POST['pseudo']) ? $_POST['pseudo'] : 'Anonyme';

// Méthode moderne avec ??
$pseudo = $_POST['pseudo'] ?? 'Anonyme';
```

L'opérateur *Nullsafe* (?->)

Introduit avec PHP 8, il permet d'appeler une méthode ou d'accéder à une propriété sur un objet qui pourrait être `null`.

Si l'objet est `null`, l'exécution s'arrête et renvoie `null` sans déclencher d'erreur fatale.

Très utile, par exemple, lors de la récupération d'objets depuis la base de données.

```php
// $user peut être un objet Utilisateur ou null (si non trouvé en base)
$user = uneFonctionQuiRenvoieUnUser(...);

// Ancienne méthode
$role = null;
if ($user !== null) {
    $role = $user->getRole();
}

// Méthode moderne avec ?->
$role = $user?->getRole();
```

Validation des données et injection de code

Validation des entrées

Il est souvent nécessaire de vérifier le format des entrées afin qu'elles correspondent à ce qui est attendu. PHP fournit une série de filtres pour cela. Voici quelques exemples :

```php
$input = "test@test.com";
if(filter_var($input, FILTER_VALIDATE_EMAIL)) {
    echo "$input correspond à une adresse email";
}
$input = "+1000";
if(filter_var($input, FILTER_VALIDATE_INT) !== false) {
    echo "$input correspond à un entier";
}
```

Il est également recommandé, en fonction du contexte, de *trimmer* les entrées :

```php
$pseudo = trim($_POST['pseudo'] ?? '');
// On valide la taille, mais on ne modifie pas le contenu !
if (strlen($pseudo) > 2 && strlen($pseudo) <= 50) {
    echo "Le pseudo a une taille valide.";
}
```

Valider n'est pas détruire !

L'objectif de la validation est de vérifier si la donnée correspond à nos attentes (type, longueur, format, …).

À l'exception de `trim()` pour enlever les espaces accidentels, il ne faut jamais altérer le contenu avec des fonctions comme `strip_tags()` ou `htmlspecialchars()` avant l'insertion en base.

La base de données doit stocker la donnée brute, et la sécurisation se fera uniquement à l'affichage.

Qu'est-ce qu'une injection de code ?

L'injection de code est une technique qui consiste à injecter des données dans une application pour en modifier le comportement. Cela se passe souvent en injectant le code malicieux via des formulaires.

Imaginez que la requête suivante soit générée en fonction d'un champ `"pseudo"` d'un formulaire :

```
$query = "SELECT * FROM User WHERE pseudo = '" . $_POST["pseudo"] . "'";
```

Si ce champ contient la chaîne `"'; DROP TABLE User --"` alors la requête ainsi générée devient :

```
$query = "SELECT * FROM User WHERE pseudo = ''; DROP TABLE User--'"
```

… ce qui a pour effet, si on ne fait pas attention, de supprimer la table `User`.

Plus de détails ici : https://fr.wikipedia.org/wiki/Injection_de_code_dans_les_applications_web

Injection de code JavaScript : Faille XSS

Dans le slide précédent, nous avons montré une injection de code SQL. Il existe d'autres types d'injections comme la faille XSS qui permet d'injecter du code JavaScript.

Nous le verrons plus tard mais un navigateur interprète et exécute le code JavaScript englobé dans un élément `<script>`.

Imaginez qu'une page affiche du texte en fonction des données d'un formulaire :

```
echo "Bonjour, " . $_POST["pseudo"] . " !";
```

Si le champ `pseudo` du formulaire contient `<script>alert('Faille XSS')</script>`, ce code est inclus dans la page HTML générée par le script PHP et est interprété par le navigateur.

Ici, la fonction `alert()` de JavaScript affiche un *pop-up* mais vous pouvez imaginer du code bien plus malicieux.

Comment se prémunir des injections de code ? (1)

La méthode `prepare()` de PDO aide à prévenir les injections SQL.

Tout se passe comme si les paramètres avaient été englobés dans des guillemets et en échappant les guillemets éventuels présents dans ces paramètres.

Si `$_POST["pseudo"]` contient `'; DROP TABLE User--` et que l'on exécute

```
$stmt = $pdo->prepare("SELECT * FROM User WHERE pseudo = :pseudo");
$stmt->execute(["pseudo" => $_POST["pseudo"]]);
```

La requête réellement exécutée pourrait être

```
SELECT * FROM User WHERE pseudo = '\'; DROP TABLE User--'
```

Une chaîne complète pour le paramètre pseudo.

plutôt que

```
SELECT * FROM User WHERE pseudo = ''; DROP TABLE User--'"
```

Deux requêtes SQL car le premier *quote* n'a pas été échappé

Comment se prémunir des injections de code ? (2)

Pour la faille XSS, il est nécessaire d'échapper (*escaping*) toute sortie en fonction du contexte désiré en échappant les caractères spéciaux et les caractères réservés à l'aide de la fonction `htmlspecialchars($string, $options, $encoding)` :

```
htmlspecialchars("<p>Bonjour</p>", ENT_QUOTES, "UTF-8")
```

donne

```
&lt;p&gt;Bonjour&lt;/p&gt;
```

Règle d'or

En résumé, pour assurer la cohérence des données et se prémunir des injections de code, il faut toujours :

1. « *Trimmer* » et valider les données à l'entrée (*filtres, tailles, …*).
   Est-ce que cela ressemble à ce que j'attends ?

2. Manipuler la base de données en utilisant des requêtes préparées avec des paramètres (`prepare()` et `execute()` de PDO).

3. Échapper les sorties via une fonction d'échappement (*escaping*). Par exemple, pour le contexte HTML :

```php
function escape($var) {
    return htmlspecialchars((string) $var, ENT_QUOTES, "UTF-8");
}
```

Redirections

Comment faire une redirection ?

Il est parfois nécessaire de rediriger le navigateur vers une certaine page.

Par exemple, si un utilisateur est connecté, on le redirige vers sa page d'accueil et sinon on le redirige vers la page de connexion (login).

Pour ce faire, il faut envoyer une réponse HTTP au navigateur dont l'en-tête contient une nouvelle URL qu'il doit suivre.

Voici une fonction que vous pouvez utiliser pour ce faire :

```php
function redirect($url, $statusCode = 303)
{
    header('Location: ' . $url, true, $statusCode);
    die();
}
```

Plus de détails ici : http://stackoverflow.com/questions/768431/how-to-make-a-redirect-in-php

Les sessions

Les sessions

Une session permet de conserver des données entre plusieurs accès d'une même instance d'un client (navigateur) à une application web. Par exemple, conserver l'identifiant de l'utilisateur connecté.

Le tableau superglobal `$_SESSION` permet de stocker ces données.

Chaque script PHP qui en a besoin doit appeler la fonction `session_start()` avant d'utiliser ce tableau. Cette fonction crée la session si elle n'existe pas encore et la restaure dans le cas contraire.

Pour clôturer une session (par exemple lors d'une déconnexion explicite), il faut écraser le tableau `$_SESSION` en lui assignant un tableau vide et appeler la fonction `session_destroy()`.

Bonne pratique : Utilisez `session_regenerate_id(true)` au moment de l'authentification pour renouveler l'identifiant de session et éviter qu'il ne soit usurpé (fixation de session).

Les cookies ont un comportement similaire aux sessions si ce n'est que les données sont stockées par le client plutôt que par le serveur et que le but est de conserver des données entre les sessions.

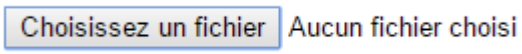
Faire la seconde (3b) et la troisième partie (3c) du tutoriel.

Uploader des fichiers

Comment *uploader* un fichier ? (1)

Dans un formulaire, on place un champ de type « `file` » :

```
<input type="file" name="icone" />
```



Pour que le navigateur sache qu'il doit envoyer un fichier, on l'indique dans l'attribut « `enctype` » de la balise `<form>` en y mettant la valeur « `multipart/form-data` », qui spécifie que le formulaire envoie des données binaires (fichier) et du texte (champs de formulaire).

```
<form method="post" action="upload.php" enctype="multipart/form-data">
    Fichier:
    <input type = "file" name = "icone">
    <br><br>
    <input type = "submit" value = "Soumettre">
</form>
```

Comment *uploader* un fichier ? (2)

Lorsque le visiteur soumet la page, les fichiers sont uploadés et placés dans un répertoire temporaire, auquel seul le script PHP pourra accéder.

Il faut ensuite contrôler le fichier puis le déplacer vers un dossier du site. On peut contrôler plusieurs choses : la taille, le type, le nom, les dimensions (si c'est une image), …

Le tableau `$_FILES` contient certaines informations sur les fichiers. Pour chaque champ de type ***file***, on peut obtenir les informations le concernant dans le tableau `$_FILES[`*`attribut_name`*`]`. On retrouve notamment les informations suivantes :

```php
<?php
$_FILES['icone']['name'];     //Le nom original du fichier, comme sur le disque du visiteur (exemple : mon_icone.png).
$_FILES['icone']['type'];     //Le type du fichier. Par exemple, cela peut être « image/png ».
$_FILES['icone']['size'];     //La taille du fichier en octets.
$_FILES['icone']['tmp_name']; //L'adresse vers le fichier uploadé dans le répertoire temporaire.
$_FILES['icone']['error'];    //Le code d'erreur, qui permet de savoir si le fichier a bien été uploadé.
?>
```

Comment *uploader* un fichier ? (3)

Contrôles du fichier :

```php
if (!isset($_FILES['icone'])) {
    $erreur = "Pas de fichier transmis.";
}
else if ($_FILES['icone']['error'] > 0) {
    $erreur = "Erreur lors du transfert.";
}
else if ($_FILES['icone']['size'] > $maxsize) {
    $erreur = "Le fichier est trop gros.";
}
else if (!in_array($_FILES['icone']['type'], ["image/jpeg", "image/png"])) {
    $erreur = "Extension incorrecte.";
}
else {
    $sizes = getimagesize($_FILES['icone']['tmp_name']);
    if($sizes[0] > $maxwidth or $sizes[1] > $maxheight) {
        $erreur = "Image trop grande.";
    }
}
```

Comment *uploader* un fichier ? (4)

Finalement, on déplace le fichier à l'aide de la fonction `move_uploaded_file()` :

```php
if(!isset($erreur)) {
    $nom = "icones/".$_FILES['icone']['name'];
    $resultat = move_uploaded_file($_FILES['icone']['tmp_name'], $nom);
    if($resultat) {
        echo "Transfert réussi.";
    }
}
```

Le répertoire de destination peut être créé dynamiquement grâce à la fonction `mkdir()` :

```
bool mkdir ( string $pathname [, int $mode = 0777 [, bool $recursive = false [, resource $context ]]] )
```

Hachage des mots de passe

Pourquoi et comment ne pas stocker un mot de passe "en clair" ? (1)

Principe général :

Pour des questions de confidentialité et de déontologie, il faut éviter de stocker en clair les mots de passe des utilisateurs dans la base de données.

Pour résoudre cela, on va stocker un code (une **empreinte**) qui sera calculé à partir du mot de passe grâce à un algorithme de calcul **non réversible** (càd qu'on ne pourra pas retrouver le mot de passe à partir du code). Ce type d'algorithme s'appelle le « ***hashing*** » ou « **hachage** ». Le résultat est un string unique et de longueur fixe que l'on appelle un « *hash code* ».

Ainsi, lors d'une phase d'authentification, on ne compare plus deux mots de passe en clair mais deux empreintes du mot de passe.

Pourquoi et comment ne pas stocker un mot de passe "en clair" ? (2)

En PHP, on peut faire appel à la fonction suivante, où le premier paramètre définit l'algorithme de hachage à utiliser (ceux-ci varient par la longueur du code retourné et par la qualité du cryptage utilisé. Voir http://php.net/manual/fr/function.hash.php) :

```
string hash ( string $algo , string $data [, bool $raw_output = false ] )
```

Exemple :

```
<?php
echo hash('ripemd160', 'Le rapide goupil brun sauta par dessus le chien paresseux.');
?>
```

```
cdb8426cb020896cea1d040e62a0f8cf9f5b4226
```



Pourquoi et comment ne pas stocker un mot de passe "en clair" ? (3)

Cette méthode n'est pas parfaite et reste « crackable ». En effet, il existe sur Internet des dictionnaires (*rainbow tables*) capables de vous retourner la chaîne en clair d'un md5(), d'un sha1() ou d'un autre algorithme standard de *hash*. Pour peu que le mot de passe original soit un mot du dictionnaire, il est fort probable que l'on puisse le retrouver dans une *rainbow table* à partir de son *hash*.

Pour éviter cela, on pourrait hacher le mot de passe en le préfixant et en le post-fixant avec deux clés secrètes (appelées « salt » ou « seed »). *Cette technique est aujourd'hui dépréciée.*




```
$PREFIX_SALT = 'orD03s;ZPpemS';
$SUFFIX_SALT = '4232dls##ZPSA';

$hash = hash("sha256", $PREFIX_SALT.'m0tD3P4ss3'.$SUFFIX_SALT);
```

Une méthode plus sécurisée serait de générer des sels aléatoires à chaque fois…

Méthode actuellement recommandée

Bcrypt est une méthode de hachage sécurisée spécifiquement conçue pour stocker les **mots de passe**. Contrairement aux méthodes de hachage classiques comme SHA-256, bcrypt inclut un facteur de "**salage**" (salt) généré **aléatoirement** et **stocké** avec le hachage. Cela permet de produire des hachages différents même pour des mots de passe identiques, ce qui rend impossible l'usage d'une *rainbow* table.

```php
$hash = password_hash("m0tD3P4ss3", PASSWORD_BCRYPT);

if(password_verify("m0tD3P4ss3", $hash)) {
    echo "mot de passe correct";
}
```

Pour une sécurité encore plus optimale, il faudrait utiliser l'algorithme ***Argon2id*** (PASSWORD_ARGON2ID), *car il est conçu pour résister efficacement aux attaques modernes par force brute et par matériel dédié (GPU). Cependant, nous ne l'utiliserons pas dans le cadre de ce cours car la version de XAMPP qui vous est fournie n'intègre pas cet algorithme*

Voir les dernières bonnes pratiques OWASP ici : https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Faire la quatrième partie du tutoriel (3d).